

Liminal State: Deferred-Persistence Lifecycles for Operation-Based CRDTs

Merkle, B.¹ Rodionov, V.²

Draft — preprint, April 2026

Abstract

CRDTs make every operation immediately permanent — eliminating coordination but creating a mismatch with interactive applications where most input is exploratory. We present *liminal state*, a construction for deferred-persistence lifecycles on op-based CRDTs. Tentative writes live in a lower-priority replica-identifier range; committed writes live in a higher-priority range that preempts the tentative ones via the partition scheme of^[1]. Either commit or a TTL-bounded abandon produces an eventually canonical document state that converges across all peers from static algorithmic rules, without runtime coordination or consensus (§1.3). The construction is in principle single-document — abandoned tentative writes can be marked undone (Yu-style^[2]) and left inert. For efficiency, a shadow-document optimization sparing peers the wire and storage costs of propagating operations that may be abandoned is described abstractly in §2.3; Appendix A documents the concrete Yjs implementation, Appendix B the encoding-level primitives used by commit, and Appendix C the awareness-based peer preview transport. The technique handles both scalar and structural (array insert/delete/splice) operations. Deployed in Plexus/Yjs.

1 The Permanence Problem

We use the term *liminal* (after the anthropological sense of a transitional, not-yet-committed state) to refer to tentative operations that exist provisionally in the CRDT’s operation log. Neither *liminal* (in this specific CRDT sense) nor the companion term *genesis* (in the sibling paper on deterministic structural entities) is established in the CRDT literature; both are coinages introduced by this paper stack.

1.1 Exploratory Input vs. Committed Intent

CRDTs conflate the two. A slider drag, an IME composition, and a drag-reorder are all *explorations* — the user is discovering a value, not declaring one. The CRDT treats each intermediate state as a permanent decision.

Saito and Shapiro^[3] define the standard lifecycle for optimistic replication: “Operations issued in the optimistic mode accept or produce tentative states, while operations issued in the pessimistic mode appear as completed in a stable state, termed committed.” OT systems implement this directly — local operations are tentative until the server confirms ordering.

CRDTs deliberately eliminated this lifecycle. Every operation is immediately permanent: inserted into the operation log, replicated to all peers, persisted to storage. This eliminates the need for a central ordering authority — a fundamental improvement. But it means CRDTs have no native concept of “I’m still exploring.”

Note: OT’s tentative operations may be *reordered* by the server — the operation’s final form is uncertain. Liminal state provides a different lifecycle: operations may be *discarded* entirely — the operation’s existence is uncertain. OT defers ordering, where liminality defers existence. The mechanism is partition-based preemption of tentative writes by their committed successors (§2), not server coordination.

1.2 Consequences in Production

Gesture explosion and write amplification. A 10-second slider drag at 60fps produces 600 CRDT operations, each a permanent entry in the operation log. Undoing the drag requires 600 undo steps — or, with typical timer-based undo-grouping heuristics, an unpredictable number of grouped steps depending on drag speed and pauses. The tldraw team states: “people would have to hit undo hundreds of times to get a shape to its previous location”^[4]. Those 600 operations also replicate to every peer and persist to every storage backend, and benchmarks show the cost: 100,000 map operations on 10 keys in one production CRDT library produce 525KB of metadata vs 271 bytes of actual

¹Independent researcher, Here.build; x.com/merkle_bonsai; ImmuneFi top-40 whitehat (ethical hacker); email: merklebonsai@pm.me

²Independent researcher, Here.build; email: v@here.build

content — a 2000:1 ratio^[5]. For continuous gestures, the amplification is pure waste: only the final value matters.

Undo granularity. Every production CRDT application hacks undo grouping:

System	Mechanism / limitation
Yjs ^[6]	Timer debounce (500ms); arbitrary, splits mid-drag pauses
Liveblocks ^[7]	<code>pause()/resume()</code> ; imperative, missed resume breaks undo permanently
tldraw ^[4]	Mark/squash; custom system, not CRDT-native
Figma ^[8]	Pause on mousedown; server-authoritative, not decentralized
Xi-editor ^[9]	Semantic group IDs; never productionized

The academic assessment: “There is currently no generally applicable undo support for CRDTs”^[2].

A related problem shows up in IME composition. During CJK input, intermediate keystrokes are tentative — the user is composing a character — but in CRDTs each keystroke becomes a permanent operation. This causes duplicate text (Tiptap #7271), raw pinyin leaking (Tiptap #7186), deleted text reappearing (Lexical #7779), and Slate being “unusable at production level for most CJK languages”^[10]. The W3C created the EditContext API partly in response^[11]. Liminal state provides a natural *architectural fit* for treating composition as tentative (enter liminality on `compositionstart`, commit on `compositionend`); this is future work requiring binding-layer integration (see C7) and is not claimed as a delivered contribution of this paper.

Structural array operations. Reordering a list during a drag requires array insert + delete. Unlike scalar map sets (which overwrite), array operations create permanent operation-log entries and tombstones. Undoing an array delete in a tombstone-based sequence CRDT cannot simply flip a delete flag — the logical position may have shifted due to concurrent operations — so a new operation must be synthesized at the current logical position^[2,12]. This is a fundamental property of tombstone-based sequence CRDTs, not an implementation quirk.

1.3 What Liminality Actually Promises

Liminality does not promise that a commit will succeed. It promises *eventual convergence* — every peer, regardless of network conditions or participation, ends up agreeing on the same canonical document state: either (a) the pre-gesture state, or (b) the committed-gesture state. Never a third alternative, never a stuck middle.

The following walkthrough shows why. Suppose two users A and B are collaborating on a document. User A enters a liminal session at time T and begins editing. Session TTL is some fixed duration — say 5 minutes.

The common thread: **every scenario converges to a deterministic final state without any runtime coordination, leader election, or consensus protocol**. This holds because three algorithmic rules are preagreed at deployment:

1. **Partition scheme** (from^[1]): writes in the committed range preempt writes in the liminal range, which preempt writes in the regular range. Priority is categorical, not negotiated.
2. **TTL value**: every peer runs the same timer against the same start time (broadcast with the session). Expiry decisions converge without communication. Clock skew tolerance is bounded by the heartbeat (~30s), far below the TTL (~5min).
3. **Per-user X stability + order preservation** (P3 of^[1]): a user reuses the same 51-bit base X across every lifecycle stage they enter. The partition is an order-preserving shift, so if $X_A > X_B$ then A wins inter-peer ties in every stage — no flipping, no inconsistent winner-relationships between stages.

Any peer applying these three rules arrives at the same answer. The runtime is their mechanical evaluation; no coordinator, leader, or consensus protocol runs at session time. This is the “consensus without consensus” sense: the rules are encoded in every peer’s code at build time rather than negotiated per session, and runtime decisions follow from static algorithmic inputs.

The companion paper^[1] establishes the partition scheme (adapting Yu et al.’s categorical-second-dimension technique to op-based CRDTs) and the inter-peer order-preservation property that scenario 6 relies on. The present paper uses that foundation to construct a tentative-with-bounded-resolution lifecycle for CRDT operations.

2 The Construction

This section describes the technique abstractly, as it would apply to any op-based CRDT with partition-range support^[1]. The concrete Yjs implementation — the shadow-document pattern, origin-based routing, and encoding-level commit primitives — is documented

Scenario	A's fate	B's fate	Canonical outcome
1. A commits at T+2min, B online throughout	A's writes land in committed range; peer broadcast completes	B sees A's preview via the preview channel, then receives the committed writes and applies them	Committed-gesture state
2. A commits at T+2min, B offline at T+1min, returns at T+10min	A's commit completes locally and is queued for sync	B reconnects, receives A's committed writes, applies them	Committed-gesture state
3. A goes offline at T+1min without committing, returns at T+3min (within TTL)	A reconnects, resumes broadcasting preview, eventually commits normally	If A's offline window exceeded the preview channel's disconnect timeout (~30s), B dropped the preview; on A's return, a fresh preview broadcast re-establishes it. Session TTL (5min from T) is unchanged by the gap.	Committed-gesture state (after A commits)
4. A goes offline at T+1min, returns at T+7min (after TTL)	A's code detects session expiry on reconnect (clock comparison against T+5min), discards tentative writes, returns to pre-gesture state	B dropped A's preview at T+5min without coordination (local TTL timer)	Pre-gesture state
5. A goes offline at T+1min, never returns	A's session is unreachable	B independently drops A's preview at T+5min; document returns to pre-gesture state	Pre-gesture state
6. A and B both editing concurrently (separate liminal sessions)	Each has own tentative workspace, own liminal clientId	Each broadcasts own preview; previews don't interfere (different peer channels)	Both commits succeed; final state determined by the committed-range tiebreaker. Per P3 of ^[1] , if $X_A > X_B$ then A wins ties in every stage — the winner is stable and predictable from static inputs.
7. A and B both go offline mid-session, neither returns	Both sessions unreachable	Every remaining peer (or the same peers on eventual reconnect) drops both sessions at T+5min	Pre-gesture state
8. A commits at T+2min, B offline at T+3min for a month	A's commit is queued for B	B reconnects after a month, receives A's committed writes (they're in B's operation log now), applies them	Committed-gesture state

in Appendix A (shadow-document mechanics), Appendix B (commit primitives), and Appendix C (peer preview transport).

2.1 Partition-Based Preemption

The convergence guarantees of §1.3 rest on the partition scheme of^[1]: tentative writes live in a lower-priority clientId range, committed writes in a higher-priority range, and the partition ensures committed preempts tentative in every tiebreaker. Abandoned tentative writes can be marked undone (Yu-style^[2]) and left inert in the operation log — they will never win a tiebreaker against subsequent committed writes at the same position. The reason: every committed write on the same peer carries a clientId in the committed range (strictly higher-valued than any liminal clientId on the same peer, by invariant 1 of §2.2 and the partition base offsets of^[1]), and inter-peer ties are resolved consistently across lifecycle stages by the order-preservation property (P3 of^[1]). A subsequent committed write therefore wins

the position regardless of whether the old tentative write is deleted, marked undone, or simply left in place.

Correctness rests entirely on the partition. The session lifecycle (§2.2) organizes when writes enter which range; the shadow-document optimization (§2.3) avoids the wire cost of propagating tentative writes that may be abandoned. Neither contributes to correctness beyond what the partition already provides.

2.2 Session Lifecycle

A liminal session is a bounded interval during which a peer's operations are tentative:

- **Enter.** The peer allocates a fresh clientId in the liminal range and designates subsequent operations as tentative. A start timestamp is recorded for TTL computation.
- **Operate.** Writes performed during the session carry the liminal clientId. Locally they are visible to the peer (optimistic UX); they may be transmitted to other peers via the preview channel

(§3) but are not merged into those peers’ persistent state.

- **Commit.** The peer’s tentative writes are rewritten under a fresh committed-range `clientId` and delivered as a single atomic update to all peers. The partition scheme guarantees the committed writes preempt the tentative originals on every peer. The peer’s original tentative writes are abandoned (see below).
- **Abandon.** The tentative writes are discarded by the peer, or — in a single-document implementation — marked undone in place. The TTL rule ensures abandonment happens even if the peer becomes permanently offline: every other peer, running the same TTL timer, independently drops the session state at the preagreed deadline.

Session-level invariants the construction maintains:

1. **Strictly increasing `clientId` per peer.** Each new session uses a higher `clientId` than the peer’s previous session. Combined with inter-peer order preservation (P3 of^[1]), this gives a deterministic total order over session commits.
2. **Committed writes are disjoint from tentative writes.** Commit does not mutate the session’s tentative writes; it produces a separate block of writes in the committed range. Abandonment handling does not need to “move” anything.
3. **Fresh `clientId` post-boundary.** After commit or abandon, the peer allocates a new `clientId` for future operations. Reusing a session’s `clientId` would produce clock gaps visible to peers.
4. **Delta atomicity.** The committed delta is a single unit — new committed-range writes and any tentative-range cleanup — applied together. A peer observing the commit sees the new canonical state without visible intermediate stages.

These invariants are abstract; the Yjs realization of each is in Appendix A (§A.4), with additional Yjs-specific invariants in §A.5.

2.3 Shadow Document as Efficiency Optimization

A single-document implementation works but is expensive: every tentative write propagates to peers at its liminal `clientId` and must be marked undone on abandon, paying wire and storage costs for operations

that will never become canonical. For a 600-op slider drag that’s abandoned, that is 600 operations replicated and persisted for nothing.

The shadow-document optimization separates tentative writes from committed writes into two CRDT containers on the originating peer:

- **Shadow** — local-only; holds tentative writes; not synchronized with remote peers.
- **Main** — the authoritative document; synchronized with remote peers; receives only committed writes.

On commit, tentative writes are extracted from shadow, rewritten to the committed range, and applied to main — which then syncs normally to peers. On abandon, tentative writes are dropped from shadow; main is untouched; peers never saw the tentative writes on the persistent channel (though they may have seen a preview via the separate preview channel of §3).

Correctness is preserved because the partition still does the work: on the originating peer, the shadow/-main split just routes tentative writes to a local-only container instead of the synchronized one. On other peers, the sequence of events is identical to the single-document case (they only ever see committed-range writes in their main document).

The optimization saves $O(\text{session size})$ operations on the sync and persistence paths per session. For abandoned sessions, that saving is the entire cost of the session. The cost paid is roughly a second-doc memory footprint, bounded by active session size rather than document lifetime (§6, C3).

Implementations on op-based CRDT runtimes other than Yjs may realize this via the shadow-document pattern or via functionally equivalent mechanisms — for instance, attaching a lifecycle tag to operations so the sync layer can filter tentative ones (mark-undone-on-abandon + sync-layer filter), or using an in-memory journal that only merges into the replicated store on commit. All that matters for correctness is that abandoned tentative writes do not reach remote peers’ persistent state, or that if they do, they are marked undone per §2.1 so subsequent committed writes preempt them via partition priority. §A.6 describes what this looks like in Loro and Automerge concretely.

3 Peer Preview

In-progress tentative writes are not visible to remote peers through the normal CRDT sync path (that would

defeat the point of tentative). Instead, a separate *preview channel* broadcasts the session’s current tentative state to peers as ephemeral awareness-like data.

3.1 Preview Channel Semantics

The preview channel is conceptually separate from the CRDT sync channel:

- **Ephemeral:** preview state auto-expires on peer disconnect; it is not persisted; it generates zero entries in any peer’s permanent operation log.
- **Per-session, per-peer:** each session broadcasts its current tentative state; receivers track session state per originating peer.
- **Idempotent supersession:** later preview updates replace earlier ones. A receiver can apply only the latest without loss.
- **Cleanup-safe:** a session-end signal (commit, abandon, or TTL expiry) causes receivers to drop their per-session preview state locally. No acknowledgment needed; the signal is mechanical, not negotiated.

In the Plexus implementation, the preview channel is the y-protocols awareness protocol with per-session state entries; see Appendix C. The construction does not depend on this specific channel — any peer-to-peer ephemeral state broadcast mechanism with per-peer keyed state works.

3.2 Receiver Protocol

A receiver maintains a local per-peer preview store, applied to the receiver’s own local-only tentative workspace (not main). On receiving a preview update for a session, the receiver:

1. Decodes the preview and applies it locally (tentative only).
2. Records the session’s identity and start timestamp.
3. On supersession (newer preview arrives), replaces the prior preview.
4. On session-end signal or TTL expiry, drops the stored preview and undoes its local application.

The receiver’s preview application operates only on the receiver’s local tentative workspace, mirroring the originator’s shadow/main split. Main is never touched by preview application.

3.3 Collective TTL

Every peer — originator and receivers — runs the same TTL timer against the same session start timestamp. At TTL expiry, each peer independently:

- **Originator:** cancels the session if not yet committed; discards the tentative workspace.
- **Receivers:** drops the per-session preview state, undoes its local application.

Collective TTL depends on approximate wall-clock agreement across peers. Clock skew below TTL causes no functional problem; skew that exceeds TTL in one direction (receiver’s clock ahead) causes premature preview expiry (safe — the originator’s commit is still honored); skew in the other direction (receiver’s clock behind) causes delayed cleanup (also safe — the preview remains idempotent with respect to eventual state).

A secondary heartbeat mechanism (the preview channel’s own disconnect timeout) bounds cleanup independently of wall-clock agreement: when a peer becomes unreachable, its preview entries expire via the channel’s disconnect handling regardless of clock.

3.4 Security Scope

Preview content is applied to the receiver’s tentative workspace without content validation (the preview is a byte blob produced by another peer). A malicious peer could inject arbitrary state into receivers’ tentative workspaces; the blast radius is bounded to tentative, never reaching main. Production deployments relaying previews through a server can validate preview bytes at the relay.

4 Structural Liminality

Scalar tentative writes (map-key overwrites, single-value updates) are straightforward: the final value in the tentative workspace is the commit payload; abandonment discards the workspace.

Structural tentative writes (sequence insert, delete, splice) are more subtle because sequence CRDTs use tombstones: a logical delete leaves a tombstone marker in the operation log, and the operation log position is part of the CRDT’s conflict-resolution state. Three cases arise:

- **Insert-only session.** The tentative workspace contains new items; the committed delta carries those items under the committed `clientId` and appends them to main.

- **Delete-only session.** The tentative workspace contains only tombstones on main items; the committed delta carries only a delete set (no new items under the liminal `clientId`). Commit applies this delete set to main.
- **Mixed session.** Inserts + deletes. The commit must deliver both: new committed-range items AND delete markers for main items that the session deleted. Additionally, tentative items that were inserted and then deleted within the same session must not appear on commit — their tombstones are local to tentative workspace cleanup.

The three-case dispatch is what the commit operation handles. In a tombstone-based sequence CRDT, undoing a sequence deletion generally cannot “flip” the delete flag because the logical position at the time of the original operation may have shifted due to concurrent edits. The undo must synthesize a new operation at the current logical position^[2,12]. This creates *ghost items* under the liminal `clientId` during abandonment — extra tentative-range items synthesized by the undo process. Cleanup requires detecting and dropping these ghosts; Appendix B (§B.4) documents the Yjs detection mechanism. The existence of ghosts is inherent to tombstone-based sequence CRDTs, not a Yjs quirk; the specific detection mechanism is runtime-specific.

5 Properties

P1. Deferred persistence. Operations are temporary by default (liminal), permanent on explicit commit. This matches interactive editing where most gestures are exploratory. The standard CRDT model makes every operation immediately permanent; liminality gates persistence on an explicit commit decision.

P2. Low-cost exploration. Tentative operations produce zero permanent entries in main on reverted gestures. Active session cost is bounded by session size; at every session boundary the tentative workspace can be rebuilt from main state, returning its memory to $O(\text{document size})$. Cost scales with *active* session size, not with the document’s lifetime.

P3. Atomic commit. The committed delta is a single binary update encoding the net effect of the entire gesture. 600 intermediate slider values $\rightarrow$ 1 committed delta. The commit is one undo step regardless of gesture duration.

P4. Structural soundness. Array insert, delete, and splice during liminal sessions produce correct committed deltas via three-case dispatch (§4). Tombstone-

based sequence CRDTs additionally require ghost-item cleanup — a generic consequence of their delete-undo semantics — whose Yjs-specific detection mechanism is in §B.4.

P5. Peer preview without persistence. In-progress gestures are visible to peers via a separate ephemeral channel. Zero permanent operations for previews. Auto-expires on disconnect or TTL.

P6. Adaptive performance. Preview broadcast frequency can respond to CPU pressure; tab-hidden suppression is a natural extension. See Appendix C for the Yjs implementation.

P7. Namespace isolation. Liminal, committed, regular, and genesis operations occupy non-overlapping `clientId` ranges^[1]. Lifecycle stage is a single range check. Filtering, routing, and priority ordering derive from the namespace.

P8. Concurrent cross-peer safety. Two peers in simultaneous liminal sessions on the same entities operate independently on their tentative workspaces. Both commit. The committed deltas carry distinct `clientIds` and merge via normal CRDT conflict resolution — with the inter-peer winner determined by the stable ordering of P3 of^[1]. No special handling needed.

P9. Eventual convergence under bounded-skew network conditions. The eight-scenario walk-through of §1.3 — online, offline, partial, delayed, long-duration, concurrent — all converge to a deterministic canonical state without runtime coordination, subject to the bounded-clock-skew assumption of C5. The three preagreed rules (partition scheme, TTL, per-user X stability) determine outcomes from static inputs; the TTL rule is the only one that depends on approximate wall-clock agreement across peers.

6 Constraints

C1. Op-based CRDT with numeric replica identifiers. The technique requires an op-based CRDT where (a) the conflict tiebreaker reads a replica identifier and (b) the identifier space can be partitioned^[1]. State-based CRDTs and position-based sequence CRDTs do not qualify.

C2. Single active session per peer. One liminal session per peer at a time. Multiple concurrent sessions would require sub-partitioning the liminal namespace, which is feasible but not implemented here.

C3. Tentative-workspace memory. Maintaining the tentative workspace (§2.3) requires roughly doubled local memory during active sessions, bounded by the active session’s operation count. At every session

boundary, the workspace can be rebuilt from main state, returning memory to $O(\text{document size})$. Cost scales with the active session, not with the document’s lifetime. The Yjs-specific realization — which document needs `gc: false`, which can retain default GC — is covered in §A.1.

C4. Preview trust boundary. Preview content is applied to receivers’ tentative workspaces without validation. Malicious previews are confined to tentative (never reach main). Hardening requires server-side validation at the preview relay.

C5. Wall-clock dependency for collective TTL. The TTL mechanism uses wall-clock comparison; large clock skew causes early or late cleanup. A secondary disconnect-based timeout bounds cleanup independently of clock.

C6. Ghost cleanup inherent to tombstone sequence CRDTs. Undoing a sequence delete in a tombstone-based sequence CRDT generally requires synthesizing a new operation, producing ghost items that must be cleaned up. This is fundamental; only the detection mechanism is runtime-specific.

C7. Binding-layer integration required for IME. Liminality provides the architecture for treating IME composition as tentative; the integration with editor bindings is not yet implemented.

C8. Host runtime must provide specific extension points. The implementation described in the appendices depends on features the host CRDT runtime must surface: a mutable replica-identifier per document, a differential-encoding API that produces updates relative to a state vector, and an undo framework exposing the equivalents of capture-boundary, delete-filter, and origin-tracking hooks. The Yjs-specific API names are in the appendices; §A.6 discusses portability to other runtimes (Loro, Automerge).

7 Related Work

Saito and Shapiro^[3] define the tentative/committed lifecycle for optimistic replication. OT’s tentative operations may be reordered by a central authority — the operation’s final form is uncertain. Liminality provides a structurally analogous but semantically distinct lifecycle: operations may be discarded — the operation’s existence is uncertain. The mechanism is local isolation rather than server coordination.

Yu, Elvinger, Ignat^[2] introduce the categorical-second-dimension technique for state-based CRDTs, encoding per-operation lifecycle state (active/undone) as a second timestamp dimension. The companion

partitioning paper^[1] adapts that technique to op-based CRDTs; liminality is an application.

GGPO rollback netcode^[13] provides the most precise structural analogy. Confirmed state = main. Predicted state = tentative workspace. Misprediction rollback = abandon. Correction = commit. GGPO assumes deterministic simulation with a single authority, where liminality operates in a decentralized CRDT; GGPO replays corrected inputs, where liminality commits new operations.

Automerge transactions^[14] provide commit/rollback within a single document, but they are synchronous (closure-scoped), while liminal sessions span arbitrary wall-clock time (drag gestures, IME composition). Automerge has no tentative-workspace isolation, no peer preview, and no structural array ghost handling.

Ink & Switch Upwelling^[15] introduces “drafts” as lightweight CRDT branches for creative privacy. The architecture is conceptually similar to shadow/main, but Upwelling uses full Automerge document clones and merge-based reconciliation. Liminality operates on the same document format with origin-based routing, using encoding-level primitives for surgical commit rather than full document merge.

Ink & Switch Patchwork^[16] explores version control on Automerge documents with branches and merges. Like Upwelling, uses `clone()` + `merge()`.

Liveblocks pause()/resume()^[7] provides explicit undo boundaries. This solves undo granularity but not write amplification (operations are still permanent), not peer preview, and not structural arrays.

Delta-state CRDTs^[17] formalize delta composition via join-semilattice operations. The encoding-level primitives liminality uses for commit (Appendix B) — `clientId` rewriting and targeted delete-set construction — are outside this formalization. They operate on the binary encoding layer, not the lattice structure.

Eg-walker^[18] (Gentle, Kleppmann) treats CRDT metadata as ephemeral computation, materialized only during merges and then discarded. Orthogonal to liminality and composable: a tentative workspace using eg-walker’s approach would not maintain persistent CRDT metadata, addressing the memory concerns of C3.

Yjs UndoManager^[6] groups operations by timer with explicit boundary calls. Issue #290 (open since 2020) requests semantic grouping. Liminality provides semantic grouping via sessions.

Companion techniques. Entity-oriented CRDT documents^[19] provide the architectural setting — a flat registry of shells with pointer-based references,

a concrete self-resolving UUID encoding (§5 of^[19]), and a runtime discipline for identity stability under undo (Appendix A of^[19]) — in which liminal entities carry 1-prefixed identifiers and merge cleanly into the reachable graph on commit. Semantic identifier partitioning^[1] provides the namespace ranges. Deterministic genesis^[20] provides convergent initialization. Liminality composes with all three but the tentative-workspace pattern itself is architecturally independent.

8 Conclusion

CRDTs made every operation permanent to eliminate coordination. For consistency this was the right tradeoff, but it is a mismatch with interactive applications where most input is exploratory rather than declarative.

Liminal state provides a visibility lifecycle for CRDT operations: deferred persistence via partition-based preemption of tentative writes by their committed successors. Tentative writes live in a lower-priority range; commit rewrites them into a higher range that preempts the originals on every peer; abandonment is a TTL-bounded collective drop. The convergence guarantee across any network or participation pattern (§1.3) follows from three preagreed algorithmic rules — the partition scheme^[1], the TTL value, and per-user identifier stability across stages — with no runtime coordination.

The Plexus/Yjs implementation uses the shadow-document optimization described in §2.3; appendices A–C document its concrete construction, commit primitives, and preview transport. The costs of the chosen implementation are real but bounded: roughly doubled memory for the tentative workspace (bounded by active session size, since the workspace can be rebuilt from main at every session boundary), Yjs-specific encoding primitives, single active session per peer, and ghost cleanup coupled to the host CRDT’s undo mechanism. Only the tentative workspace runs with `gc: false` — main retains standard GC and tombstones do not accumulate on the persisted path. In return, a 600-tick slider drag becomes one committed delta instead of 600 permanent operations; array reorders stop accumulating permanent ghosts; undo granularity is determined by the application rather than by a timer heuristic; and peer preview generates no permanent operations.

A Yjs Shadow-Document Implementation

This appendix documents the concrete Yjs realization of the abstract tentative-workspace construction of §2.3. Implementations on other op-based CRDT runtimes need equivalent mechanisms; §A.5 discusses what’s portable and what’s Yjs-specific.

A.1 Two Y.Doc Setup

Liminal state uses two Y.Doc instances per editing session:

- **Shadow document** (Y.Doc) — the tentative workspace. Application reads and writes target shadow. Configured with `gc: false` to preserve the in-session undo history (both the user-facing UndoManager and the liminal UndoManager need retention for their stack items). Memory cost is bounded by the active session — §6 (C3) describes the session-boundary rebuild pattern.
- **Main document** (Y.Doc) — the committed store. Syncs to peers via providers. Persisted to storage. Retains Yjs’s default GC (`gc: true`) — tombstones do not accumulate on the persisted path.

A.2 Origin-Based Forwarding

Normal (non-liminal) writes applied to shadow forward to main via origin-based routing. During a liminal session, writes tagged with the liminal origin are held on shadow and not forwarded:

```
Normal write:  user -> shadow (SHADOW_TO_MAIN) ->
               forwarded to main -> synced to peers
Liminal write: user -> shadow (LIMINAL) -> held on
               shadow, not forwarded
Commit:       extractCommittedDelta(shadow, main) ->
               apply to main -> synced to peers
Revert:       UndoManager.undo() on shadow -> liminal
               Items removed; main untouched
```

The shadow→main forwarding filter is a whitelist: only known-safe origins (SHADOW_TO_MAIN, COMMIT_DELTA, FROM_SHADOW, the main UndoManager instance) are forwarded. All other origins — including liminal writes, per-peer preview origins, and the liminal UndoManager — are blocked.

A.3 Namespace Scheme (Yjs ClientId Ranges)

Liminal state uses the partitioning scheme of^[1] to assign `clientId` ranges to lifecycle stages:

[0, 2 ⁵¹)	Regular	- normal user operations
[2 ⁵¹ , 2 ⁵²)	Liminal	- tentative session operations
[2 ⁵² , 3x2 ⁵¹)	Committed	- committed liminal deltas
[3x2 ⁵¹ , 2 ⁵³)	Genesis	- deterministic structural entities

The namespace is encoded in the two leading bits of the 53-bit safe integer space. In the shadow-document implementation (this paper), namespace conversion is arithmetic: $\text{committedId} = \text{liminalId} + 2^{51}$, and given a committed clientId the originating liminal session is recoverable as $\text{liminalId} = \text{committedId} - 2^{51}$. This arithmetic reversibility is specific to the shadow-document variant, where commit *rewrites* tentative ops from liminal-range to committed-range clientIds ; in a single-document variant, commit would write new ops directly at committed-range clientIds without a structural link back to the originating session.

A.4 Session Lifecycle (Yjs Realization)

Enter. Increment the shadow document’s clientId (strictly increasing, within the liminal range). Set the transaction origin to LIMINAL. All subsequent writes are held on shadow.

Operate. User interacts normally. Each operation is a CRDT write on shadow under the liminal clientId . A liminal UndoManager captures these.

Commit. The critical sequence, composed from the Appendix B primitives: `extractCommittedDelta` (§B.2), `buildDeleteSet` (§B.3), and `getMaxClock / applyUpdate` (standard Yjs).

```

Algorithm: commitLiminality()
1.  limId <- shadow.clientId
2.  committedId <- limId + 251
3.  delta <- extractCommittedDelta(shadow, main.
    stateVector, limId, committedId)
4.  mainUndoManager.stopCapturing()
5.  applyUpdate(main, delta, COMMIT_DELTA)    // main
    mutated - syncs to peers
6.  mainUndoManager.stopCapturing()
7.  if delta contains liminal-range structs:
8.    preUndoClock <- getMaxClock(shadow, limId)
9.    liminalUndoManager.undoAll()            //
    shadow cleanup
10. postUndoClock <- getMaxClock(shadow, limId)
11. if postUndoClock > preUndoClock:          // ghost
    detection
12.   applyUpdate(shadow, buildDeleteSet(limId,
    preUndoClock, postUndoClock))
13. shadow.clientId <- limId + 1              // fresh
    clientId
14. clear liminal state flags

```

For pure-delete sessions (line 7: no liminal structs, only deletions), skip UndoManager undo entirely —

the committed delta’s delete set handles cleanup via `main→shadow` forwarding.

Revert. Undo the liminal session on shadow. Main is untouched. Fresh clientId after.

Crash recovery. Crash at any point during commit either completes the commit (from main’s perspective, if step 5 executed) or loses only the current liminal session (if step 5 did not execute). Main is the ground truth; shadow is ephemeral.

A.5 Yjs-Specific Implementation Invariants

Beyond the abstract session invariants of §2.2, the Yjs implementation maintains several platform-specific invariants:

1. **Apply-before-undo.** Apply committed delta to main before undoing liminal Items on shadow. The delta’s positional references (`origin/rightOrigin`) are valid only in the pre-undo state.
2. **Liminal UndoManager forwarding block.** The liminal UndoManager’s undo operations must not forward to main. The whitelist filter (§A.2) ensures this.
3. **Full delta encoding.** The committed delta is `mergeUpdates([rewrittenStructs, liminalDeleteSet])` — both new Items and liminal state cleanup in a single binary.
4. **ClientId isolation from setup writes.** The clientId is incremented on `enterLiminality`, not before — ensuring setup operations (genesis, lazy containers) use the regular clientId .
5. **Main UndoManager ignoreRemoteMapChanges.** The main UndoManager must be configured with `ignoreRemoteMapChanges: true`. Without this, committed deltas (which use distinct clientIds per session) are classified as “remote” by the UndoManager’s conflict detection, causing redo to silently fail after multiple commits.
6. **Undo boundary isolation.** `stopCapturing()` on the main UndoManager before AND after applying the committed delta (steps 4, 6 of the commit algorithm). Without this, the committed delta may be merged with adjacent undo groups.

A.6 Portability Beyond Yjs

The tentative-workspace pattern generalizes to any op-based CRDT runtime that provides three capabilities. Summary table:

Loro is the most tractable target: numeric `PeerID` maps cleanly onto the partition scheme^[1], `export_from` gives differential encoding, and Loro’s *Ephemeral Store* provides analogous awareness-channel semantics for previews. The primary gap is undo — Loro’s undo is application-managed; implementing the liminality commit/revert cleanup requires building the equivalent of a tracked-origins-scoped undo manager on top of Loro’s changes API.

Automerger requires substantially more adaptation. The `actorId` is a string and not freely mutable within a single document; a liminality port would likely use a secondary Automerger document as the shadow (giving it its own `actorId`). Automerger’s `getChanges(heads)` provides differential encoding. The namespace partitioning^[1] would need to adapt to string `actorIds` via prefix bytes rather than numeric range arithmetic — tractable but a different analysis.

Ghost-item cleanup (§B.4) is tombstone-dependent; any tombstone-based sequence CRDT faces the same problem. Detection mechanism varies by runtime; the principle (track clock deltas across `undoAll`; delete-set-cleanup new items) is general.

Under realistic effort estimates: Loro port is days-to-weeks of focused implementation; Automerger port is research-level (weeks-to-months) due to the `actorId`-mutability gap.

B Encoding-Level Commit Primitives (Yjs)

The commit operation uses three primitives that operate on Yjs’s binary encoding. These are not lattice-theoretic operations — they are protocol-level transformations on Yjs’s specific binary format. They compose with the standard delta join (`Y.mergeUpdates`) but are outside the delta-state CRDT formalization^[17].

All three operate in $O(\text{session size})$ — proportional to the number of Items created during the liminal session, not the total document size.

B.1 ClientId Rewriting

Given the shadow document’s struct store containing Items under `clientId L` (liminal), produce an encoded

update where those Items appear under `clientId C` (committed), with all structural references remapped.

The core operation is `withRewrittenClientId(doc, fromId, toId, callback)` — a reusable primitive that temporarily remaps a `clientId` in the struct store:

```
withRewrittenClientId(doc, L, C, callback):
1. structs <- doc.store.clients.get(L)
2. doc.store.clients.delete(L)
3. doc.store.clients.set(C, structs)
4. For each Item in structs:
   - Save original {id, origin, rightOrigin}
   - Rewrite item.id.client: L -> C
   - Rewrite item.origin.client: L -> C (if origin references L)
   - Rewrite item.rightOrigin.client: L -> C (if rightOrigin references L)
5. result <- callback()
6. Restore all saved values and swap clients Map key back
7. Return result
```

The rewrite is temporary in-place mutation with restoration in a `finally` block. Safe under JavaScript’s single-threaded execution model; requires the callback (typically `Y.encodeStateAsUpdate`) to be side-effect-free (no observers, no transactions).

Shared primitive. `withRewrittenClientId` is also used by deterministic genesis^[20] for extracting individual client vectors during the content-hash phase. The same encoding-level operation serves two independently-motivated features — liminality commit and genesis content addressing — a convergence discovered during implementation.

B.2 extractCommittedDelta

```
extractCommittedDelta(shadow, mainStateVector, L, C):
1. deleteSet <- buildDeleteSetUpdate(L, 0, maxClock(L))
2. commitDelta <- withRewrittenClientId(shadow, L, C,
   () => Y.encodeStateAsUpdate(shadow,
   mainStateVector))
3. Return Y.mergeUpdates([commitDelta, deleteSet])
```

The result is a single binary update carrying (a) the session’s writes under the committed `clientId` and (b) a delete set for the original liminal `clientId`. Applying this to a peer’s main document both adds the committed writes and (via the delete set) cleans up any preview application the peer had done under the liminal `clientId`.

B.3 Targeted Delete Set Construction

Given a `clientId` and a clock range, produce a CRDT update containing only a delete set — no struct blocks:

```
buildDeleteSetUpdate(clientId, clockStart, length):
```

Capability	Why needed	Yjs	Loro	Automerge
Mutable replica ID per document	Enter a session with a fresh ID in the liminal range; post-commit, allocate a new one	<code>doc.clientID = ...</code>	<code>peer_id</code> mutation on <code>LoroDoc</code>	Not natively mutable; requires indirection or <code>actorId</code> reassignment at transaction boundaries
Differential encoding against a state vector	<code>extractCommittedDelta</code> needs “encode only structs not yet known to main”	<code>Y.encodeStateAsUpdate(doc, sv)</code>	<code>export_from(VersionVector)</code>	<code>getChanges(heads)</code> — returns changes since heads, equivalent modulo encoding
Undo framework extension points	Shadow-local undo; capture-boundary separation at commit; delete-filter to protect creation shells; origin-tracking to avoid forwarding liminal writes	<code>UndoManager</code> (<code>trackedOrigins</code> , <code>stopCapturing</code> , <code>deleteFilter</code>)	No native undo manager — requires library-level undo with manual origin tracking	Automerge has no built-in undo manager — requires manual implementation using heads/changes

1. Encode 0 struct clients (empty struct block)
2. Encode 1 delete set entry: `{clientId, clock: clockStart, length}`
3. Return encoded update

Applied to a document, this marks exactly the specified Items as deleted without touching any other state.

B.4 Ghost Item Cleanup

When Yjs’s `UndoManager` undoes sequence deletions, it creates NEW Items to “restore” deleted elements rather than flipping delete flags on existing Items — because the logical position of the deleted element may have shifted due to concurrent operations. These ghosts have the liminal `clientId` but clock values beyond the session’s original range.

Detection: compare `getMaxClock(shadow, liminalClientId)` before and after the liminal `UndoManager`’s `undoAll`. If the clock advanced, the `UndoManager` created new Items. Build a targeted delete set for the ghost range and apply it.

The detection is conservative: it only fires when new Items appear under the liminal `clientId` during undo. This is correct because no other code creates Items under that `clientId` between the clock snapshot and the undo (the session is single-threaded and the `clientId` is isolated). If Yjs changes `UndoManager` behavior to flip delete flags instead of creating new Items, the detection becomes a no-op (safe degradation).

C Yjs Awareness-Based Preview Transport

This appendix documents the concrete Yjs realization of the abstract preview channel of §3.

C.1 Awareness Field Encoding

Liminal previews are broadcast via the y-protocols awareness protocol^[21]:

```
awareness field "liminal": [height: number, startSec:
                             number, base64delta: string]
```

- **height** — monotonic session counter (disambiguates successive sessions)
- **startSec** — wall-clock start time for collective TTL
- **base64delta** — current liminal state as an encoded Yjs update

The awareness channel is ephemeral: states auto-expire on disconnect (~30s timeout), are not persisted, and generate zero entries in the permanent operation log.

C.2 Adaptive Broadcast

Broadcast frequency adapts to CPU pressure via the `PressureObserver` API:

- **Nominal** → high-frequency updates (smooth preview)
- **Serious** → reduced frequency
- **Critical** → minimal updates (preserve responsiveness)

`visibilitychange` stops broadcast entirely when the tab is hidden. We are not aware of other collaborative systems that adapt preview frequency to device performance — Figma broadcasts at fixed 30fps^[8], Liveblocks throttles presence at fixed 100ms^[7].

C.3 Peer Preview Receiver

On receiving a preview: decode the base64 delta, apply to the local shadow document with a per-peer symbol origin, track for cleanup. On supersession or expiry: undo via per-peer UndoManager.

Race condition: The awareness clear (fast, awareness channel) may arrive before the committed delta (slower, CRDT sync). During the gap, the receiver sees the pre-gesture state. This is transient — the committed delta is idempotent and produces the correct final state when it arrives.

C.4 Collective TTL (Yjs)

All peers independently expire a liminal session after 5 minutes from `startSec`. This assumes approximately synchronized wall clocks — clock skew beyond TTL causes premature expiration (clock behind) or delayed cleanup (clock ahead). The awareness protocol’s 30-second heartbeat timeout provides a secondary safety net regardless of clock agreement.

References

- [1] Plexus Project. Adapting categorical CRDT lifecycle to op-based CRDTs via replica-identifier partitioning. Preprint, 2026.
- [2] Weihai Yu, Victorien Elvinger, and Claudia-Lavinia Ignat. A generic undo support for state-based CRDTs. In *Proc. OPODIS*, 2019.
- [3] Yasushi Saito and Marc Shapiro. Optimistic replication. *ACM Computing Surveys*, 2005.
- [4] tldraw. Multiplayer undo/redo. <https://tldraw.dev>, 2024.
- [5] Yjs Authors. Yjs utility (YKeyValue). <https://github.com/yjs/y-utility>, 2024.
- [6] Yjs Authors. Yjs UndoManager. <https://docs.yjs.dev/api/undo-manager>, 2024.
- [7] Liveblocks. How to build undo/redo in a multiplayer environment. <https://liveblocks.io/blog>, 2024.
- [8] Evan Wallace. How figma’s multiplayer technology works. <https://figma.com/blog>, 2019.
- [9] Xi-editor. CRDTs. <https://xi-editor.io/docs/crdt.html>, 2020.
- [10] Slate. Fix editing with IMEs. GitHub issue [ianstormtaylor/slate#4127](https://github.com/ianstormtaylor/slate/issues/4127), 2024.
- [11] W3C. EditContext api explainer. <https://w3c.github.io/editing/docs/EditContext/explainer.html>, 2024.
- [12] Yjs Authors. Yjs INTERNALS. <https://github.com/yjs/yjs/blob/main/INTERNALS.md>, 2024.
- [13] Tony Cannon. GGPO: Good game peace out. <https://ggpo.net>, 2009.
- [14] Automerge. Transactions. <https://automerge.org/automerge/api-docs/>, 2024.
- [15] Ink & Switch. Upwelling: Combining real-time collaboration with version control. <https://inkandswitch.com>, 2024.
- [16] Ink & Switch. Patchwork. <https://inkandswitch.com/patchwork/>, 2024.
- [17] Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 2018.
- [18] Joseph Gentle and Martin Kleppmann. Collaborative text editing with eg-walker. In *Proc. EuroSys*, 2025. arXiv:2409.14252.
- [19] Plexus Project. Entity-oriented CRDT documents: Architecture, identity, and structural liveness. Preprint, 2026.
- [20] Plexus Project. Deterministic genesis: Coordination-free structural initialization for operation-based CRDTs. Preprint, 2026.
- [21] Yjs Authors. y-protocols awareness. <https://github.com/yjs/y-protocols>, 2024.